

计算机科学与技术学院

系统开发工具基础

第三周实验报告

学生姓名： 杜铭昊

学号： 25020007021

授课教师： 周小伟

实验环境： Ubuntu Linux 虚拟机

报告内容： 课堂第 9-12 题与 10 个扩展实例

实验日期： 2026 年 9 月 1 日

目录

1 实验环境与证据规范	3
2 第 9 题：从源码构建并安装 Wheel	3
2.1 原理	3
2.2 步骤 1：建立 src 布局	3
2.3 步骤 2：构建制品	3
2.4 步骤 3：干净环境安装	3
2.5 步骤 4：目录外运行	4
2.6 结果与反思	4
3 第 10 题：可验证的智能体修复循环	5
3.1 原理	5
3.2 步骤 1：增加失败测试	5
3.3 步骤 2：给出受约束提示	5
3.4 步骤 3：最小修复与人工检查	5
3.5 步骤 4：复测与 AI 记录	5
3.6 结果与反思	6
4 第 11 题：把协作材料改成可执行信息	7
4.1 原理	7
4.2 步骤 1：重写 Issue	7
4.3 步骤 2：重写提交信息	7
4.4 步骤 3：重写评审意见	7
4.5 结果与反思	7
5 第 12 题：PyTorch 线性回归训练循环	8
5.1 原理	8
5.2 步骤 1：构造可复现数据	8
5.3 步骤 2：补全 200 轮训练	8
5.4 步骤 3：评估和验收	8
5.5 结果与反思	9
6 10 个相关扩展实例	10
6.1 实例 1：Wheel 结构与元数据	10
6.2 实例 2：真实 CLI 退出码基线	11
6.3 实例 3：可重复构建	12
6.4 实例 4：修复版干净安装	13
6.5 实例 5：Issue 质量门禁	14
6.6 实例 6：训练可重复性	15
6.7 实例 7：梯度累积	16
6.8 实例 8：eval 与 no_grad	17
6.9 实例 9：RECORD 完整性	18
6.10 实例 10：学习率稳定性	19
7 GitHub 分阶段提交记录	20

8 综合结论与错误反思

23

1 实验环境与证据规范

实验全部在 Ubuntu 虚拟机实测，Windows 负责同步与 GitHub 推送。使用 Python 3.12.13、build 1.6.0、pytest 9.1.1、PyTorch 2.6.0+cpu、Git 和 Tectonic。课堂 9–12 题逐步记录；10 个实例保留原理、过程、结果和反思。仓库为<https://github.com/ddd666j/system-tools-week3>。

2 第 9 题：从源码构建并安装 Wheel

2.1 原理

Wheel 是可安装 ZIP 制品，`pyproject.toml` 声明构建后端、发行名称和命令入口。干净环境只从生成 wheel 安装，可以证明包不依赖源码目录。

2.2 步骤 1：建立 src 布局

创建 `src/greetlab/__init__.py`、`cli.py` 和 `pyproject.toml`，发行名替换为 `greetlab-25020007021`，入口为 `sdt-greet=greetlab.cli:main`。

2.3 步骤 2：构建制品

```
ls -lh dist
```

成功生成 `sdist` 与 `py3-none-any.whl`。隔离构建首次因网络无法下载 `setuptools` 停止，改用已配置课程环境的 `--no-isolation` 完成。

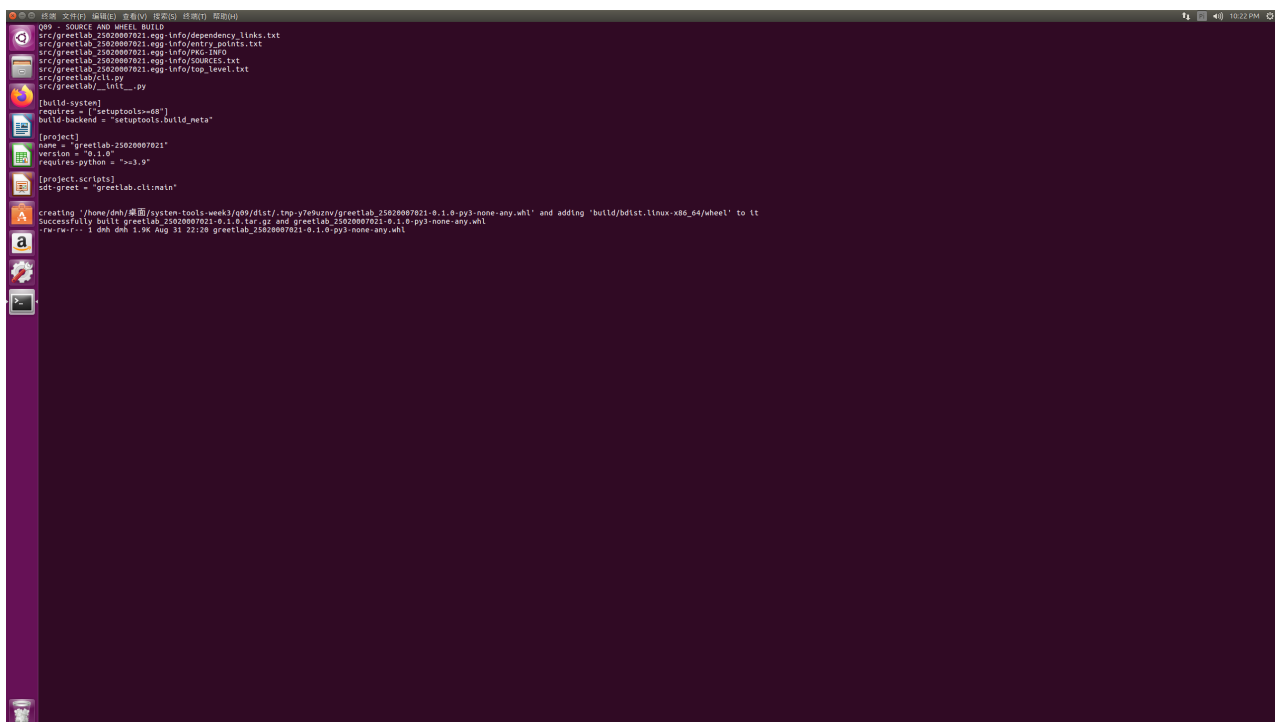


图 1: 第 9 题：源码结构、项目配置与 wheel 构建

2.4 步骤 3：干净环境安装

新建 `clean-venv`，使用 `--no-index --find-links dist` 只从本地 wheel 安装，版本为 0.1.0。

2.5 步骤 4：目录外运行

切换到/tmp 运行 `sdt-greet --name 25020007021`，输出 `Hello, 25020007021!`。

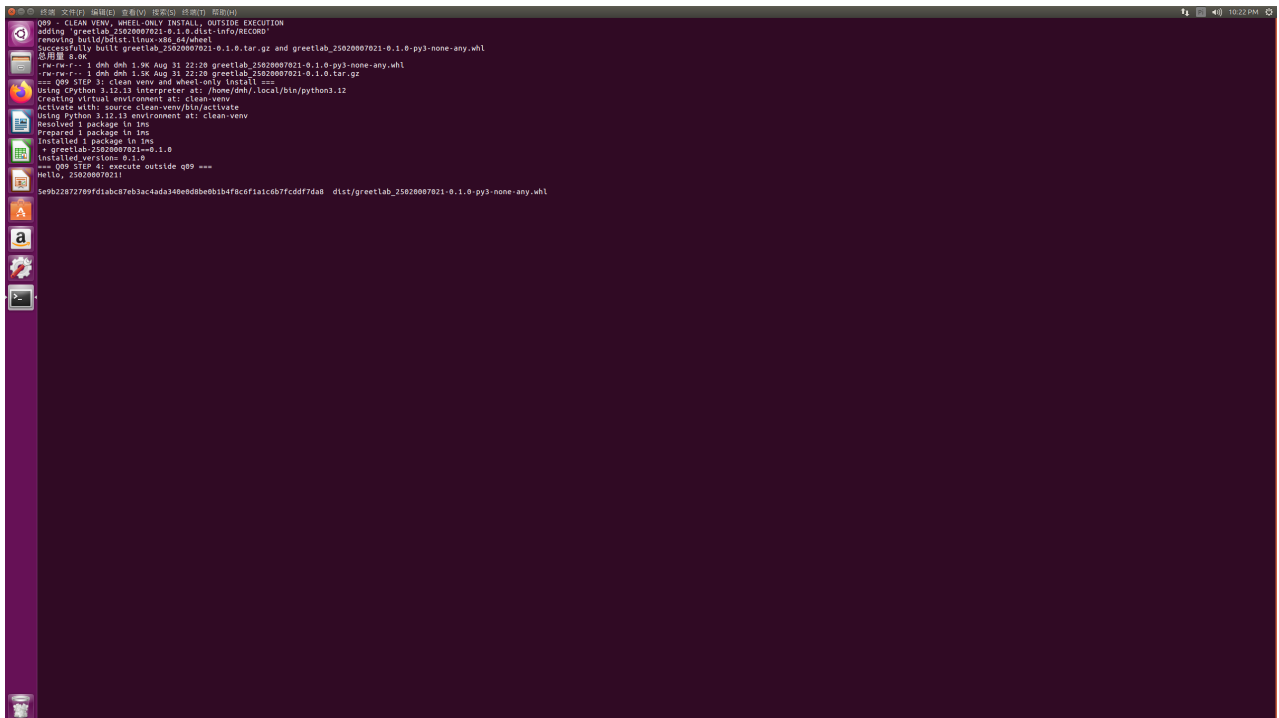


图 2: 第 9 题：干净安装与源码目录外执行

2.6 结果与反思

构建、安装和入口均通过。反思：构建成功不等于交付成功，必须在干净环境并离开源码目录验证。

3 第 10 题：可验证的智能体修复循环

3.1 原理

测试驱动修复先把需求变成失败测试，再实施最小修改、检查 diff 并复测；智能体建议必须接受人工审查。

3.2 步骤 1：增加失败测试

复制第 9 题，为空白姓名编写 `pytest.raises(SystemExit)` 并断言退出码 2。原实现输出 Hello, !，测试以 DID NOT RAISE SystemExit 失败。

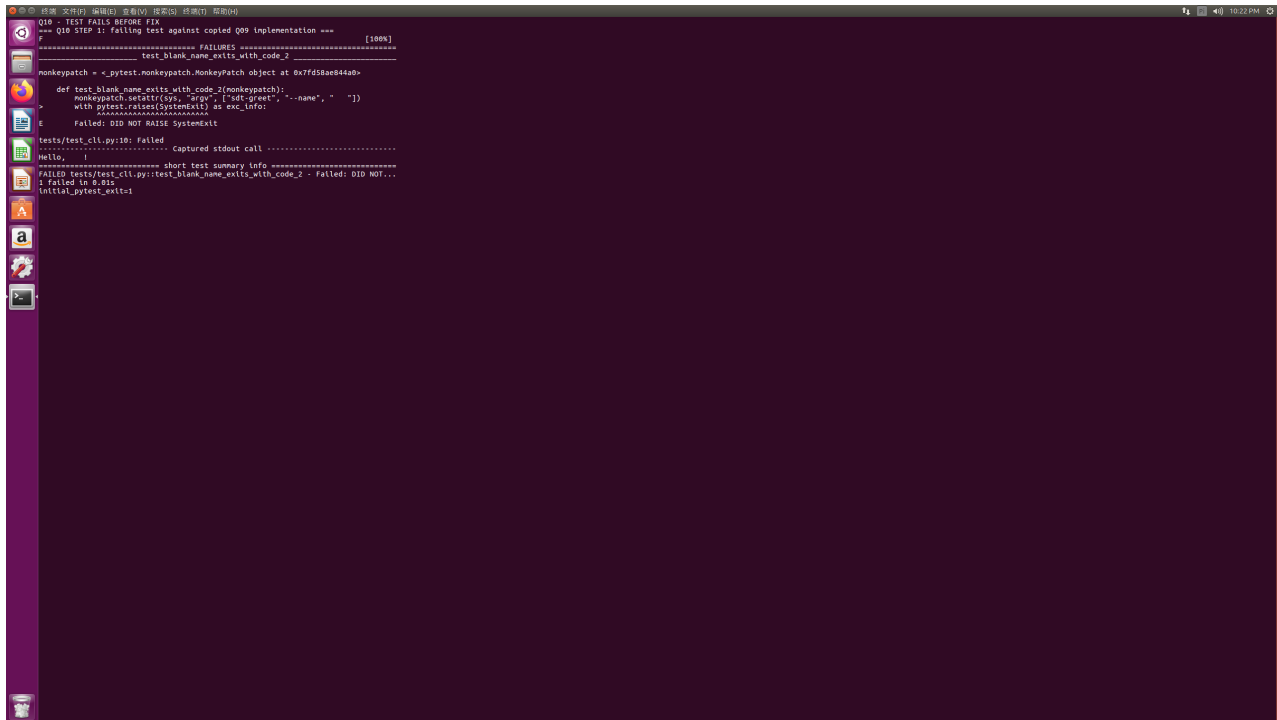


图 3: 第 10 题：修复前失败测试

3.3 步骤 2：给出受约束提示

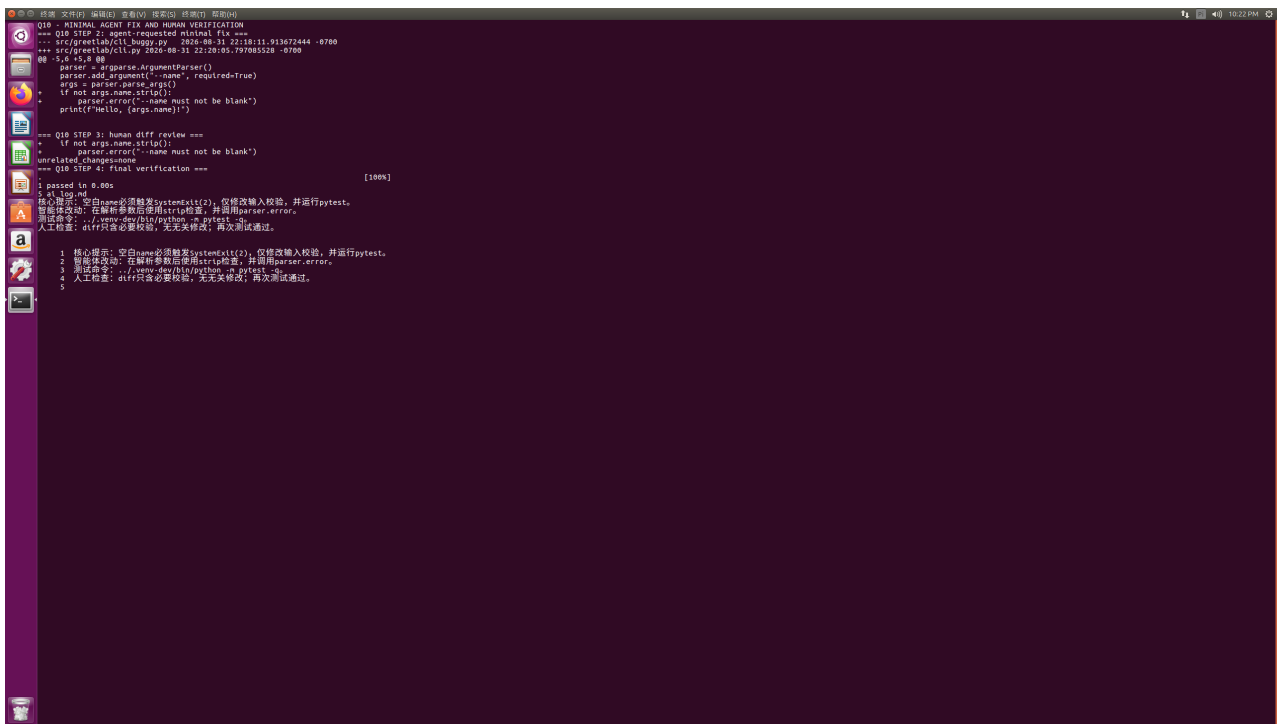
提示限定：空白 name 必须 `SystemExit(2)`，只改输入校验，运行 `pytest`，不得无关重构。

3.4 步骤 3：最小修复与人工检查

解析后用 `args.name.strip()` 判断，为空调用 `parser.error`。diff 只新增两行逻辑，无无关修改。

3.5 步骤 4：复测与 AI 记录

最终 1 passed; ai_log.md 用 5 行记录提示、修改、命令和人工验证。



```
Q10 - MINIMAL AGENT FIX AND HUMAN VERIFICATION
=== Q10 STEP 2: agent-requested minimal fix ===
src/gremlab/cil.py 2025-08-31 22:20:05.913672444 -0700
src/gremlab/cil.py 2025-08-31 22:20:05.797055528 -0700

5,4 # fix me
parser = argparse.ArgumentParser()
parser.add_argument('--name', required=True)
args = parser.parse_args()
* if not args.name.strip():
*     parser.error('--name must not be blank')
*     print("Hello, {args.name}")

=== Q10 STEP 3: human diff review ===
* if not args.name.strip():
*     parser.error('--name must not be blank')
unrelated_changes:none
=== Q10 STEP 4: final verification ===

1 passed in 0.00s [100%]

$ ./log.md
核心提示：空白name必须触发SystemExit(), 你修改输入校验, 并运行pytest.
智能体改动：在解析参数后使用strip检查, 并调用parser.error.
测试命令：./test.py --name python --pytest
人工检查：diff只告必要校验, 无无关修改, 再次测试通过.

1 核心提示：空白name必须触发SystemExit(), 你修改输入校验, 并运行pytest.
2 智能体改动：在解析参数后使用strip检查, 并调用parser.error.
3 测试命令：./test.py --name python --pytest
4 人工检查：diff只告必要校验, 无无关修改, 再次测试通过.
5
```

图 4: 第 10 题：最小 diff、人工复查与最终测试

3.6 结果与反思

失败证据、修复 diff 和通过证据形成闭环。反思：不能只记录“智能体已完成”，必须保存目标、约束、测试和人工结论。

4 第 11 题：把协作材料改成可执行信息

4.1 原理

高质量 Issue 应让他人无需追问即可复现；提交信息解释问题与方案；评审意见明确行为、风险、动作和严重程度。

4.2 步骤 1：重写 Issue

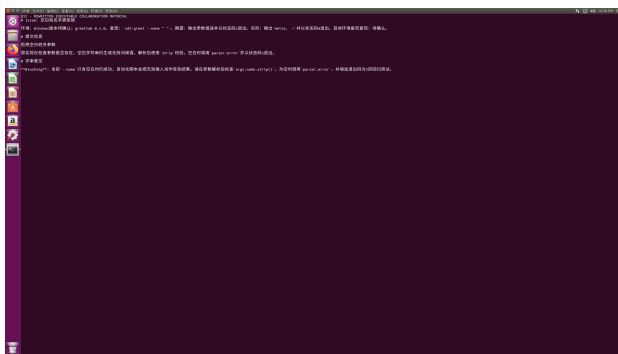
写明 Windows 版本待确认、greetlab 0.1.0、复现命令、期望状态码 2、实际状态码 0，未知信息标为“待确认”。

4.3 步骤 2：重写提交信息

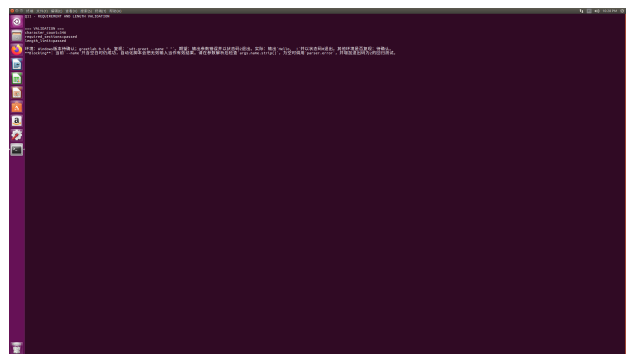
标题“拒绝空白姓名参数”使用祈使语气；正文说明原实现只检查参数存在，方案是 strip 校验并调用 parser.error。

4.4 步骤 3：重写评审意见

标注 Blocking，指出无效输入被自动化流程误判的风险，并建议增加退出码 2 回归测试。



第 11 题：完整协作材料



346 字符及字段验收

4.5 结果与反思

全文 346 字符，环境、复现、期望、实际、待确认和 Blocking 均通过自动检查。反思：简洁不是省略关键事实，而是删除无助于执行的模糊表达。

5 第 12 题: PyTorch 线性回归训练循环

5.1 原理

PyTorch 梯度默认累积, 每轮必须依次 `zero_grad`、前向计算、`backward` 和 `step`。评估时调用 `eval` 并进入 `no_grad`, 避免构建计算图。

5.2 步骤 1: 构造可复现数据

固定种子 20260907, 生成 101 个 x 与 $y=3x-1$, 模型为 `nn.Linear(1,1)`, SGD 学习率 0.1。

5.3 步骤 2: 补全 200 轮训练

每轮清零梯度、计算 MSE、反向传播和更新参数, 没有直接赋值 `weight` 与 `bias`。



```
Q12 - PYTORCH CPU TRAINING LOOP
import torch
from torch import nn

torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

model.train()
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y).item()
    weight = model.weight.item()
    bias = model.bias.item()
    print(f"final_loss={final_loss:.10f}")
    print(f"weight={weight:.10f}")
    print(f"bias={bias:.10f}")

if final_loss > 0.001:
    raise SystemExit(f"final loss did not meet the requirement")
```

图 5: 第 12 题: 完整 CPU 训练循环

5.4 步骤 3: 评估和验收

训练后 `model.eval()`, 在 `torch.no_grad()` 中得到最终损失 0、`weight` 约 2.999997、`bias` 约 -1.000000, 损失小于 0.001。



```
Q12 - DETERMINISTIC TRAINING RESULT
final_loss=0.000000000
weights=0.9999997
bias=-1.0000000

=== Q12 STEP 3: result validation ===
loss_before_0=True
parameters_learned=True

5: torch.manual_seed(20269907)
16: opt.zero_grad()
17: loss.backward()
18: opt.step()
19: model.eval()
21: with torch.no_grad():
```

图 6: 第 12 题: 学习参数与损失验收

5.5 结果与反思

模型通过梯度下降学习到目标参数。反思: `eval` 控制层行为, `no_grad` 控制自动求导, 两者不能互相替代。

6 10 个相关扩展实例

6.1 实例 1: Wheel 结构与元数据

把 wheel 作为 ZIP 读取，验证发行名称、版本、代码和命令入口，并保存 SHA-256。7 个成员和全部关键元数据通过。

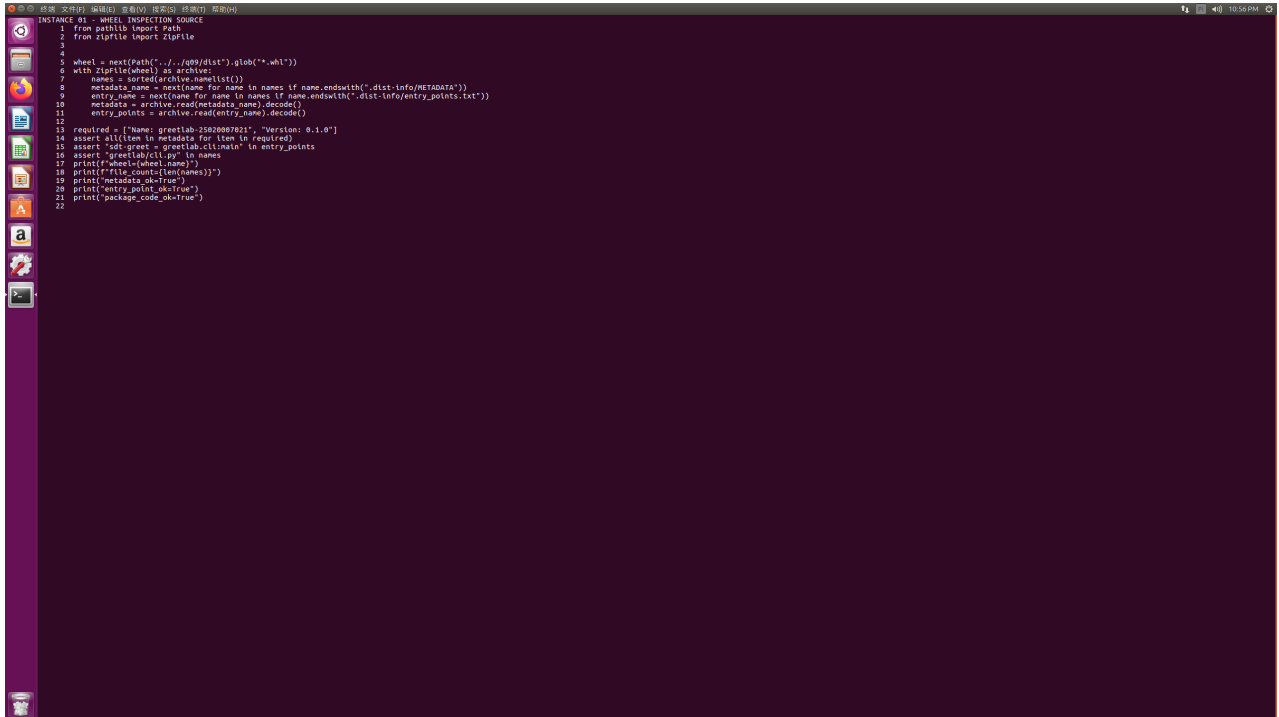


图 7: 实例 1: Wheel 检查程序

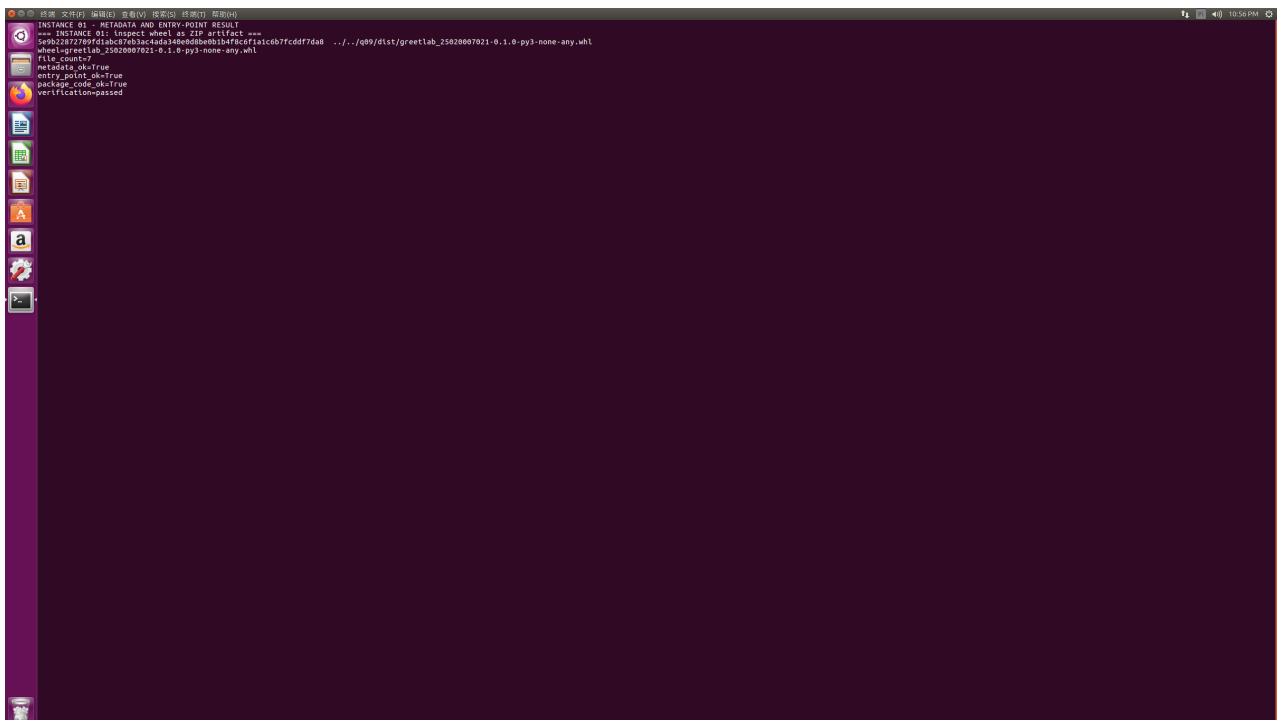
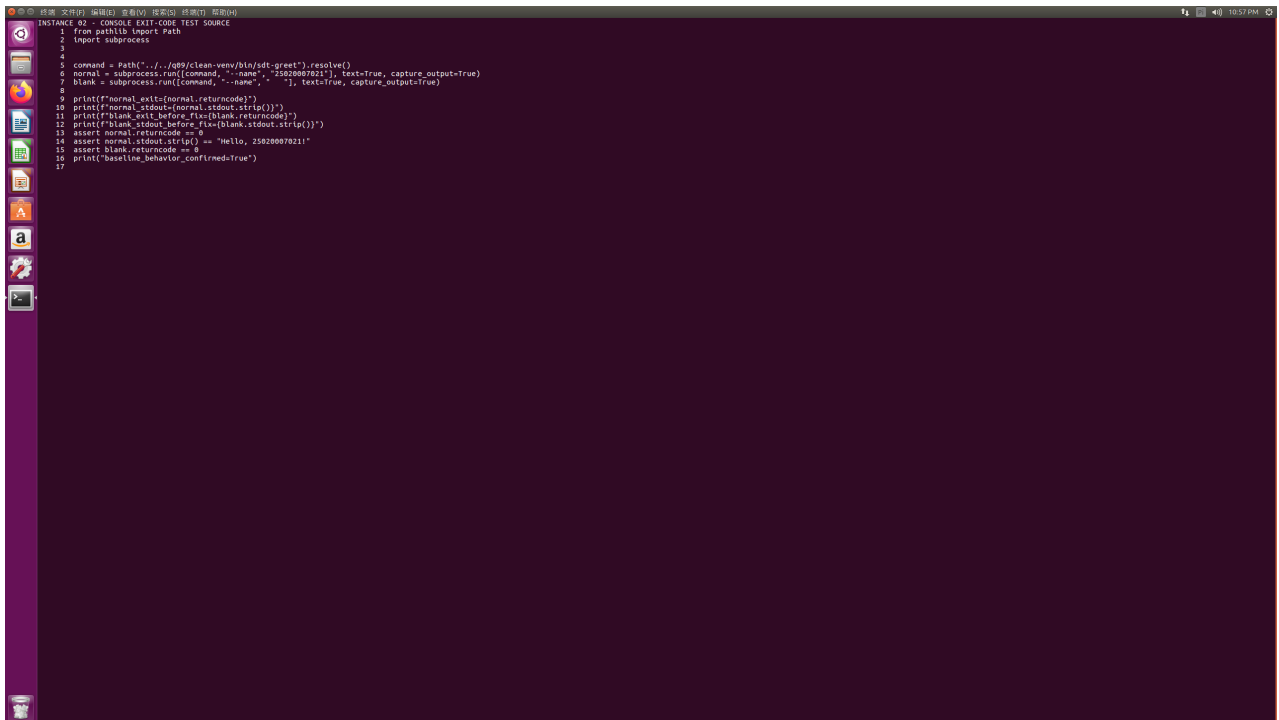


图 8: 实例 1: 元数据与入口通过

6.2 实例 2：真实 CLI 退出码基线

用 subprocess 调用干净环境入口；正常姓名返回 0，空白姓名在修复前也错误返回 0，成功从进程边界复现缺陷。



```
INSTANCE 02 - CONSOLE EXIT-CODE TEST SOURCE
1 from pathlib import Path
2 import subprocess
3
4
5 command = Path("../q9/clean-venv/bin/sdt-greet").resolve()
6 normal = subprocess.run([command, "--name", "55020087021"], text=True, capture_output=True)
7 blank = subprocess.run([command, "--name", ""], text=True, capture_output=True)
8
9 print(f"normal_exit:{normal.returncode}")
10 print(f"normal_stdout:{normal.stdout.strip()}")
11 print(f"blank_exit_before_fix:{blank.returncode}")
12 print(f"blank_stdout_before_fix:{blank.stdout.strip()}")
13 assert normal.returncode == 0
14 assert normal.stdout.strip() == "Hello, 55020087021!"
15 assert blank.returncode == 0
16 print("baseline_behavior_confirmed=True")
17
```

图 9: 实例 2：子进程测试



```
INSTANCE 02 - NORMAL AND BLANK-NAME BASELINE
=== INSTANCE 02: real console exit-code baseline ===
normal_exit=0
normal_stdout=Hello, 55020087021!
blank_exit_before_fix=0
blank_stdout_before_fix=Hello,
baseline_behavior_confirmed=True
verificationpassed
```

图 10: 实例 2：基线行为

6.3 实例 3：可重复构建

固定 SOURCE_DATE_EPOCH 独立构建两次，两份 wheel SHA-256 一致且 cmp 逐字节通过。

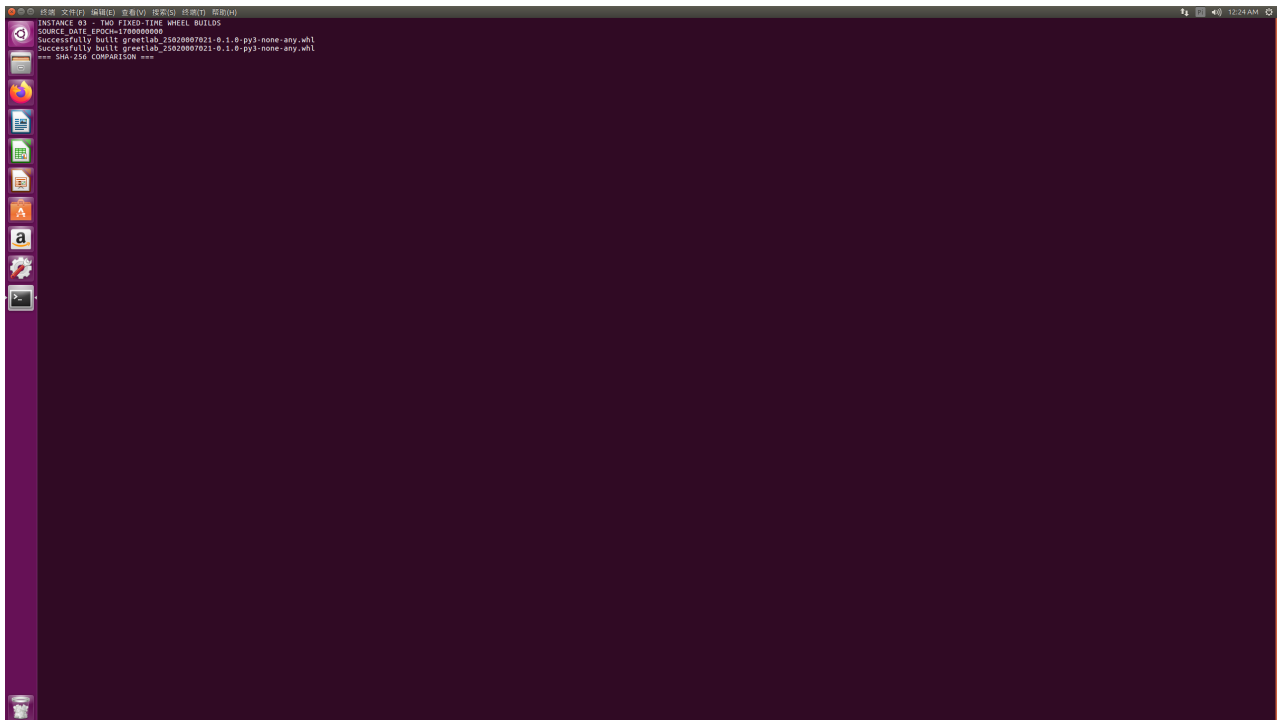


图 11: 实例 3：两次构建

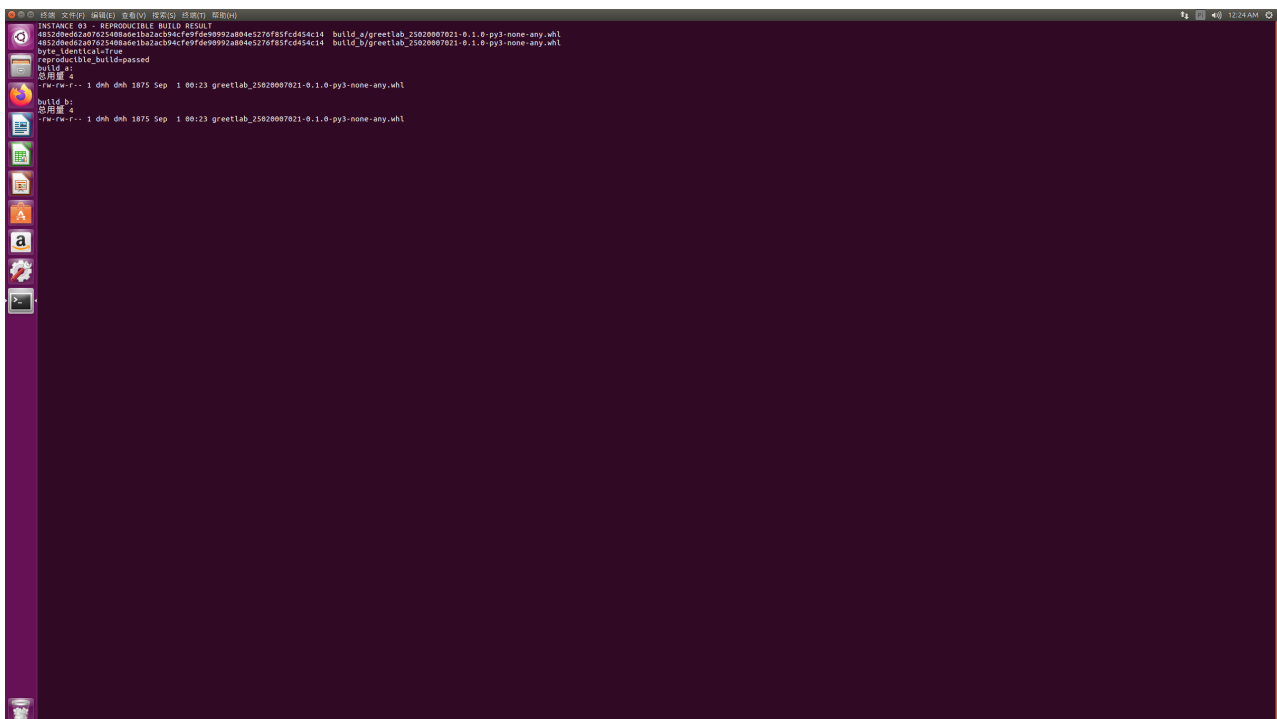
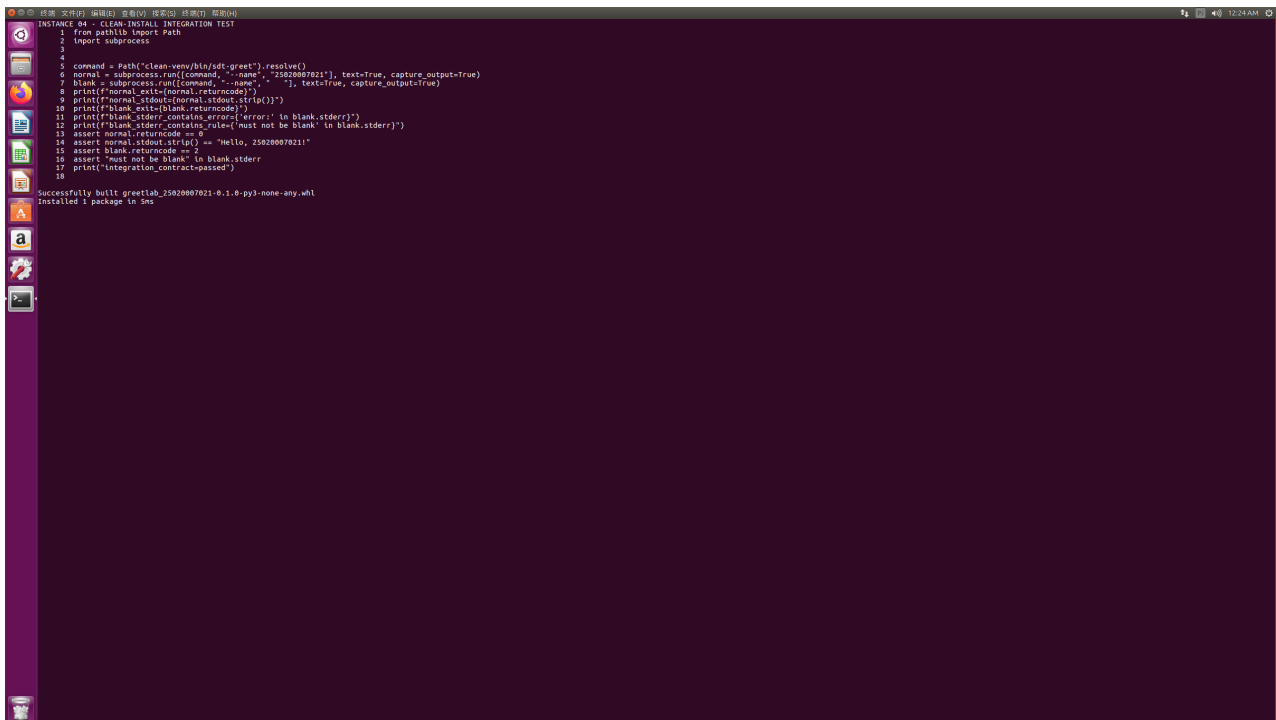


图 12: 实例 3：哈希一致

6.4 实例 4：修复版干净安装

重新构建第 10 题 wheel，在全新环境安装；正常姓名返回 0，空白姓名返回 2 且 stderr 包含规则。



```
INSTANCE 04 - CLEAN-INSTALL INTEGRATION TEST
1 from pathlib import Path
2 import subprocess
3
4
5 command = Path("clean-venv/bin/greet").resolve()
6 normal = subprocess.run([command, "--name", "25620087021"], text=True, capture_output=True)
7 blank = subprocess.run([command, "--name", ""], text=True, capture_output=True)
8 print(f"normal_exit={normal.returncode}")
9 print(f"normal_stdout={normal.stdout.strip()}")
10 print(f"blank_exit={blank.returncode}")
11 print(f"blank_stderr_contains_error={'error:' in blank.stderr}")
12 print(f"blank_stderr_contains_rule={'must not be blank' in blank.stderr}")
13 assert normal.returncode == 0
14 assert normal.stdout.strip() == "Hello, 25620087021!"
15 assert blank.returncode == 2
16 assert "must not be blank" in blank.stderr
17 print("integration_contract_passed")
18
Successfully built greetlab_25620087021-0.1.0-py3-none-any.whl
Installed 1 package in 5ms
```

图 13: 实例 4：集成测试



```
INSTANCE 04 - REAL CLI EXIT-CODE CONTRACT
=== REAL CONSOLE CONTRACT ===
normal_exit=0
normal_stdout=Hello, 25620087021!
blank_exit=2
blank_stderr_contains_error=True
blank_stderr_contains_rule=True
integration_contract_passed
```

图 14: 实例 4：CLI 契约通过

6.5 实例 5: Issue 质量门禁

模糊 Issue 缺少 5 个字段并返回 1；修订版 139 字符且全部字段通过。反思：协作材料也可自动验收。

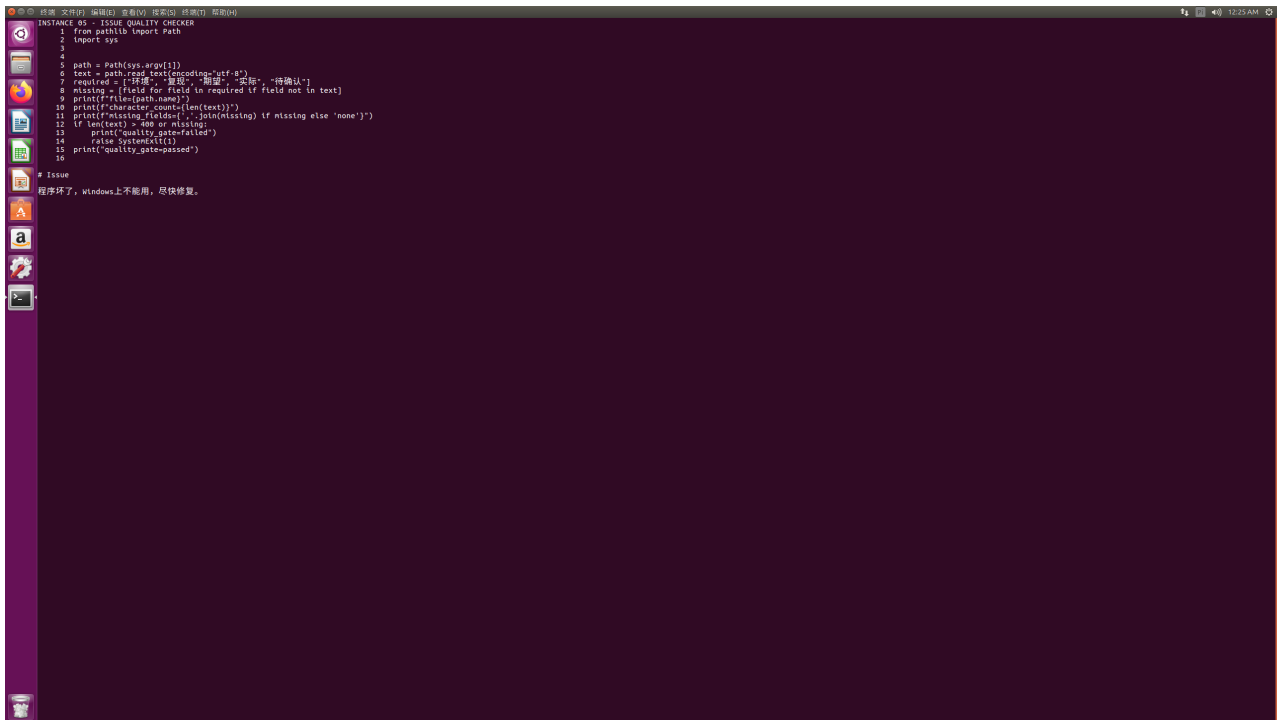


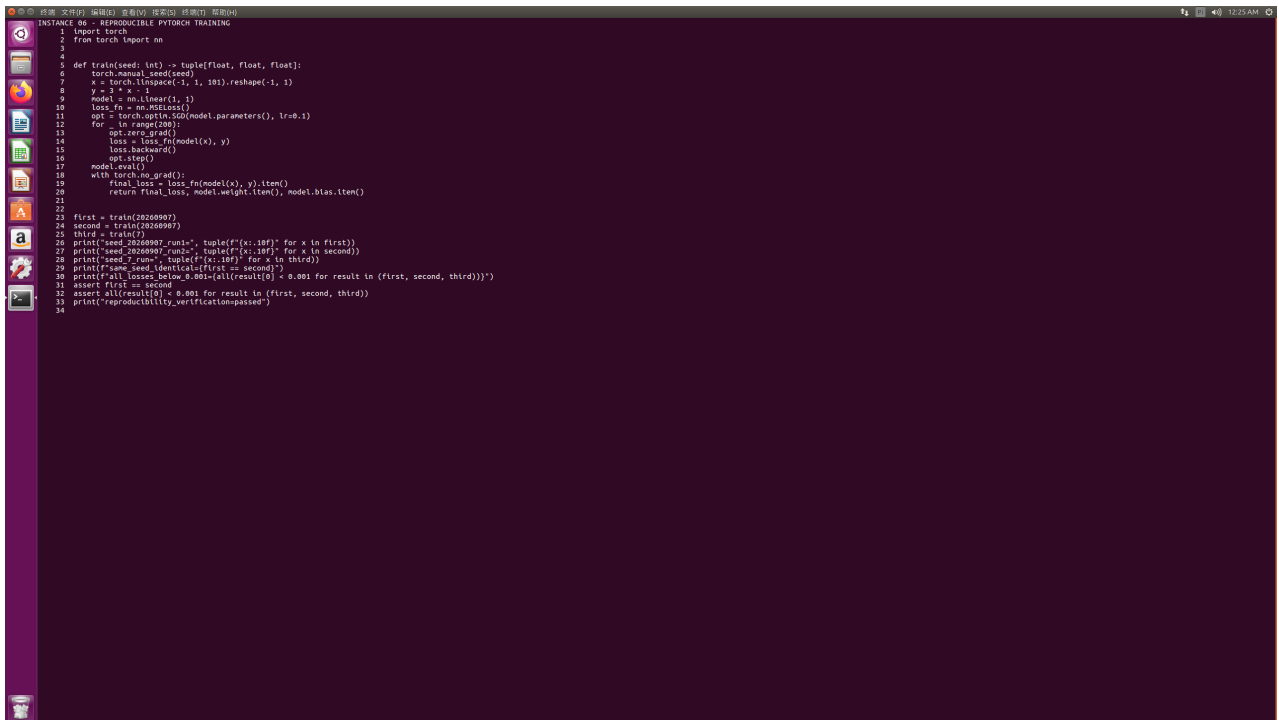
图 15: 实例 5: 门禁程序



图 16: 实例 5: 失败与通过

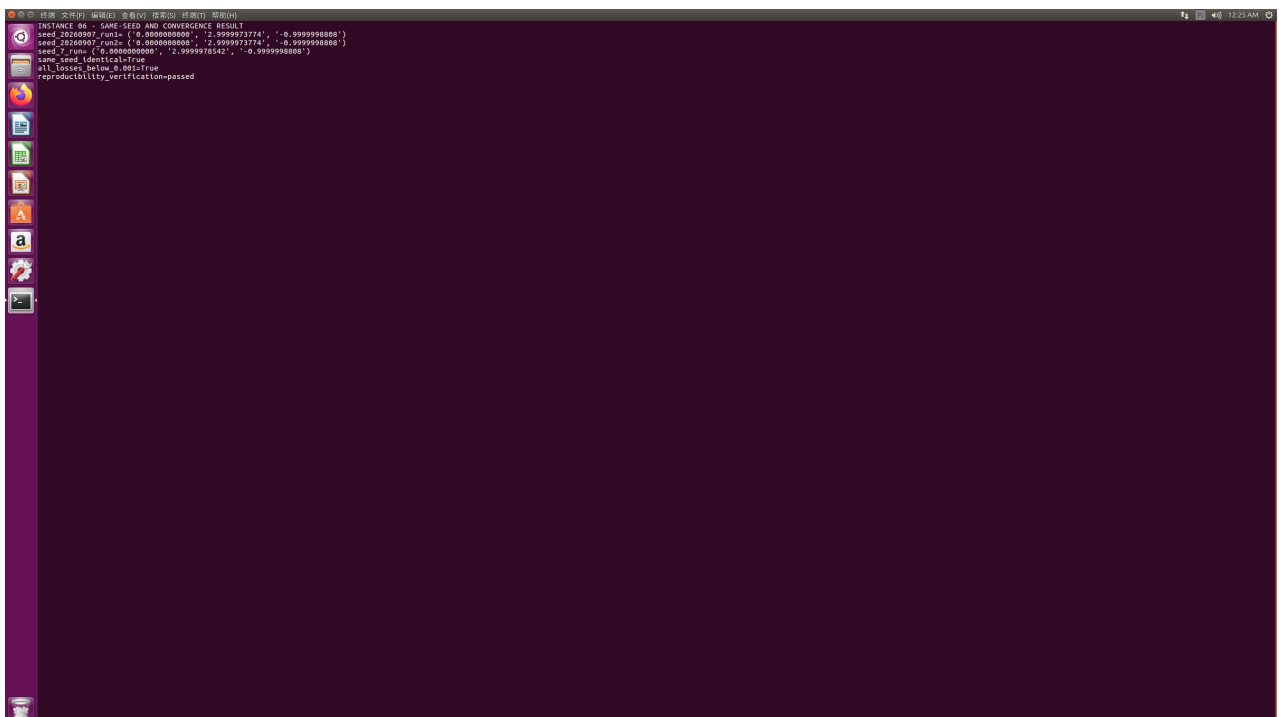
6.6 实例 6：训练可重复性

同种子两次训练结果完全一致；另一种子也收敛且损失小于 0.001。



```
1 import torch
2 from torch import nn
3
4
5 def train(seed: int) -> tuple[float, float, float]:
6     torch.manual_seed(seed)
7     x = torch.linspace(-1, 1, 101).reshape(-1, 1)
8     y = 1 * x - 1
9     model = nn.Linear(1, 1)
10    loss_fn = nn.MSELoss()
11    opt = torch.optim.Adam(model.parameters(), lr=0.1)
12    for _ in range(200):
13        opt.zero_grad()
14        loss = loss_fn(model(x), y)
15        loss.backward()
16        opt.step()
17    model.eval()
18    with torch.no_grad():
19        final_loss = loss_fn(model(x), y).item()
20        return final_loss, model.weight.item(), model.bias.item()
21
22
23 first = train(20260907)
24 second = train(20260907)
25 third = train(2)
26 print("seed_20260907_run1=", tuple(f"{x:.10f}" for x in first))
27 print("seed_20260907_run2=", tuple(f"{x:.10f}" for x in second))
28 print("seed_2_run=", tuple(f"{x:.10f}" for x in third))
29 print(f"same_seed_identical={first == second}")
30 print(f"all_losses_below_0.001={all(result[0] < 0.001 for result in (first, second, third))}")
31 assert first == second
32 assert all(result[0] < 0.001 for result in (first, second, third))
33 print("reproducibility_verification-passed")
34
```

图 17: 实例 6：训练封装

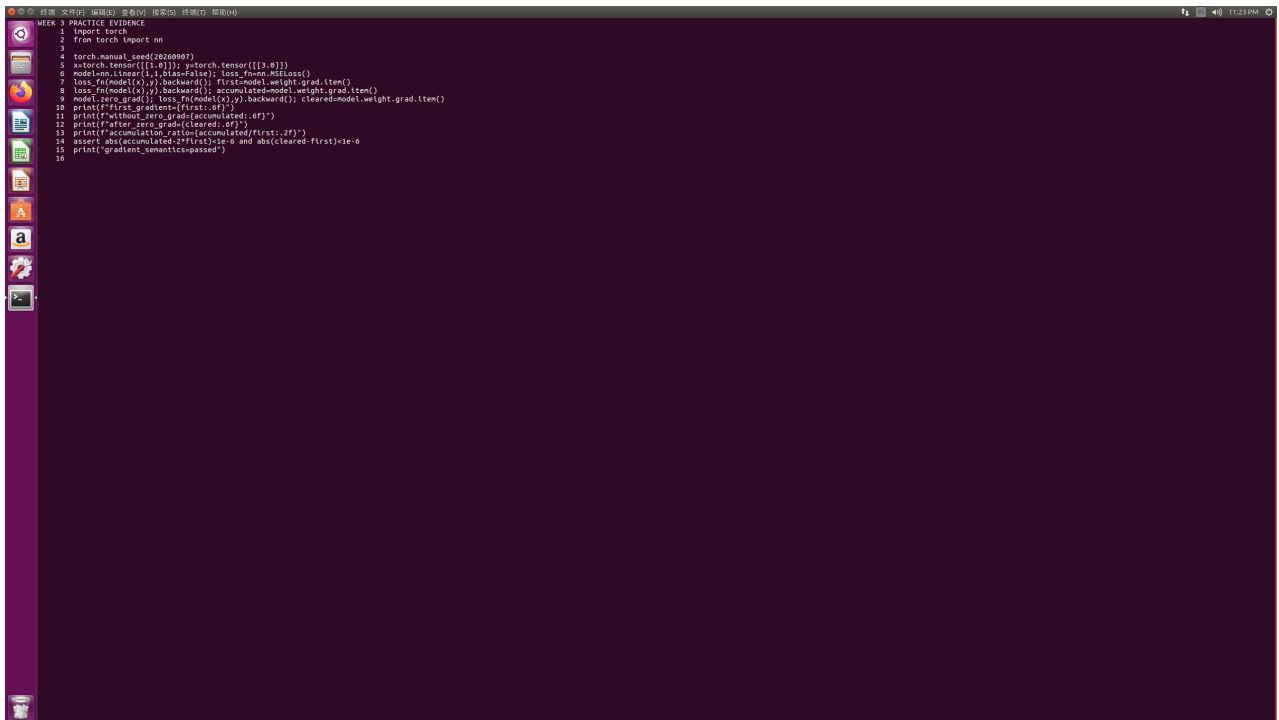


```
1 import torch
2 from torch import nn
3
4
5 def train(seed: int) -> tuple[float, float, float]:
6     torch.manual_seed(seed)
7     x = torch.linspace(-1, 1, 101).reshape(-1, 1)
8     y = 1 * x - 1
9     model = nn.Linear(1, 1)
10    loss_fn = nn.MSELoss()
11    opt = torch.optim.Adam(model.parameters(), lr=0.1)
12    for _ in range(200):
13        opt.zero_grad()
14        loss = loss_fn(model(x), y)
15        loss.backward()
16        opt.step()
17    model.eval()
18    with torch.no_grad():
19        final_loss = loss_fn(model(x), y).item()
20        return final_loss, model.weight.item(), model.bias.item()
21
22
23 first = train(20260907)
24 second = train(20260907)
25 third = train(2)
26 print("seed_20260907_run1=", tuple(f"{x:.10f}" for x in first))
27 print("seed_20260907_run2=", tuple(f"{x:.10f}" for x in second))
28 print("seed_2_run=", tuple(f"{x:.10f}" for x in third))
29 print(f"same_seed_identical={first == second}")
30 print(f"all_losses_below_0.001={all(result[0] < 0.001 for result in (first, second, third))}")
31 assert first == second
32 assert all(result[0] < 0.001 for result in (first, second, third))
33 print("reproducibility_verification-passed")
34
```

图 18: 实例 6：重复性结果

6.7 实例 7：梯度累积

连续两次 backward 且不清零时梯度为 2 倍；zero_grad 后恢复首次值。首次脚本因命令分组缺空格报错，修正后通过。



```
MEK 3 PRACTICE EVIDENCE
1 import torch
2 from torch import nn
3
4 torch.manual_seed(28269907)
5 x=torch.tensor([1.0]); y=torch.tensor([3.0])
6 model=nn.Linear(1,1,bias=False); loss_fn=nn.MSELoss()
7 loss_fn(model(x),y).backward(); first=model.weight.grad.item()
8 loss_fn(model(x),y).backward(); accumulated=model.weight.grad.item()
9 model.zero_grad(); loss_fn(model(x),y).backward(); cleared=model.weight.grad.item()
10 print(f'first_gradient={first:.6f}')
11 print(f'without_zero_grad={accumulated:.6f}')
12 print(f'after_zero_grad={cleared:.6f}')
13 print(f'accumulation_ratio={accumulated/first:.2f}')
14 assert abs(accumulated-2*first)<1e-6 and abs(cleared-first)<1e-6
15 print("gradient_semantics-passed")
16
```

图 19: 实例 7：梯度实验

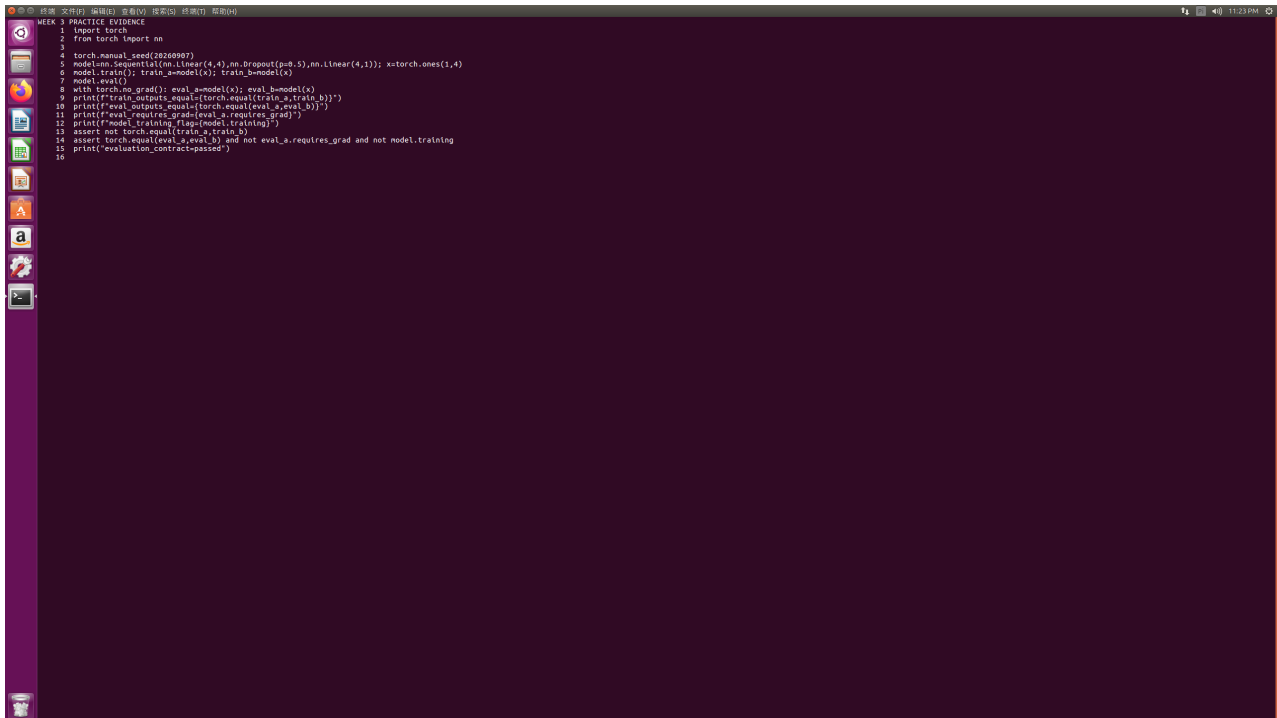


```
MEK 3 PRACTICE EVIDENCE
first_gradient=-0.398807
without_zero_grad=-0.797613
after_zero_grad=-0.398807
accumulation_ratio=2.00
gradient_semantics-passed
```

图 20: 实例 7：累积比例 2.00

6.8 实例 8: eval 与 no_grad

训练模式 Dropout 输出不同；评估模式输出一致、不跟踪梯度且 training 标志为 False。



```
1 import torch
2 from torch import nn
3
4 torch.manual_seed(42050507)
5 model = nn.Sequential(nn.Linear(4, 4), nn.Dropout(p=0.5), nn.Linear(4, 1)); x = torch.ones(1, 4)
6 model.train(); train_a = model(x); train_b = model(x)
7 model.eval()
8 with torch.no_grad(): eval_a = model(x); eval_b = model(x)
9 print(f"train_outputs_equal={torch.equal(train_a, train_b)}")
10 print(f"eval_outputs_equal={torch.equal(eval_a, eval_b)}")
11 print(f"eval_requires_grad={eval_a.requires_grad}")
12 print(f"model_training_flag={model.training}")
13 assert torch.equal(train_a, train_b)
14 assert torch.equal(eval_a, eval_b) and not eval_a.requires_grad and not model.training
15 print('evaluation_contract-passed')
```

图 21: 实例 8: 评估程序

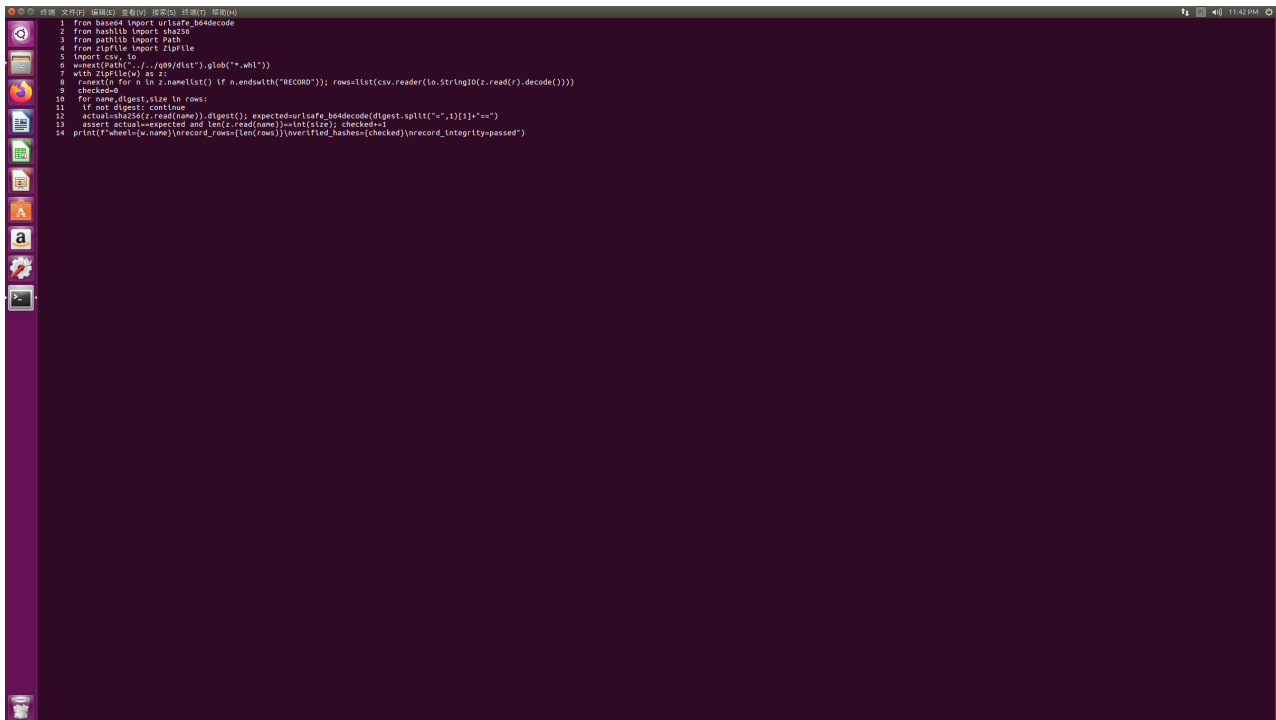


```
train_outputs_equal=True
eval_outputs_equal=True
eval_requires_grad=False
model_training_flag=False
evaluation_contract-passed
```

图 22: 实例 8: 评估契约通过

6.9 实例 9: RECORD 完整性

重新计算 wheel 中 6 个已记录文件的 SHA-256 和大小, 7 条 RECORD 记录全部一致。



```
1 from base64 import urlsafe_b64decode
2 from hashlib import sha256
3 from pathlib import Path
4 from zipfile import ZipFile
5 import csv, io
6 wheel_path = Path("../q99/dist").glob("*.whl")
7 with ZipFile(wheel_path[0]) as z:
8     rows = list(csv.reader(io.StringIO(z.read("RECORD").decode())))
9     checked = 0
10    for name, digest_size in rows:
11        if not digest_size: continue
12        actual = sha256(z.read(name)).digest(); expected = urlsafe_b64decode(digest.split("-")[1][1:-1])
13        assert actual == expected and len(z.read(name)) == int(digest.split("-")[0])
14        checked += 1
15    print(f"wheel={wheel_path[0]} record_rows={len(rows)} verified_hashes={checked} record_integrity=passed")
```

图 23: 实例 9: RECORD 校验

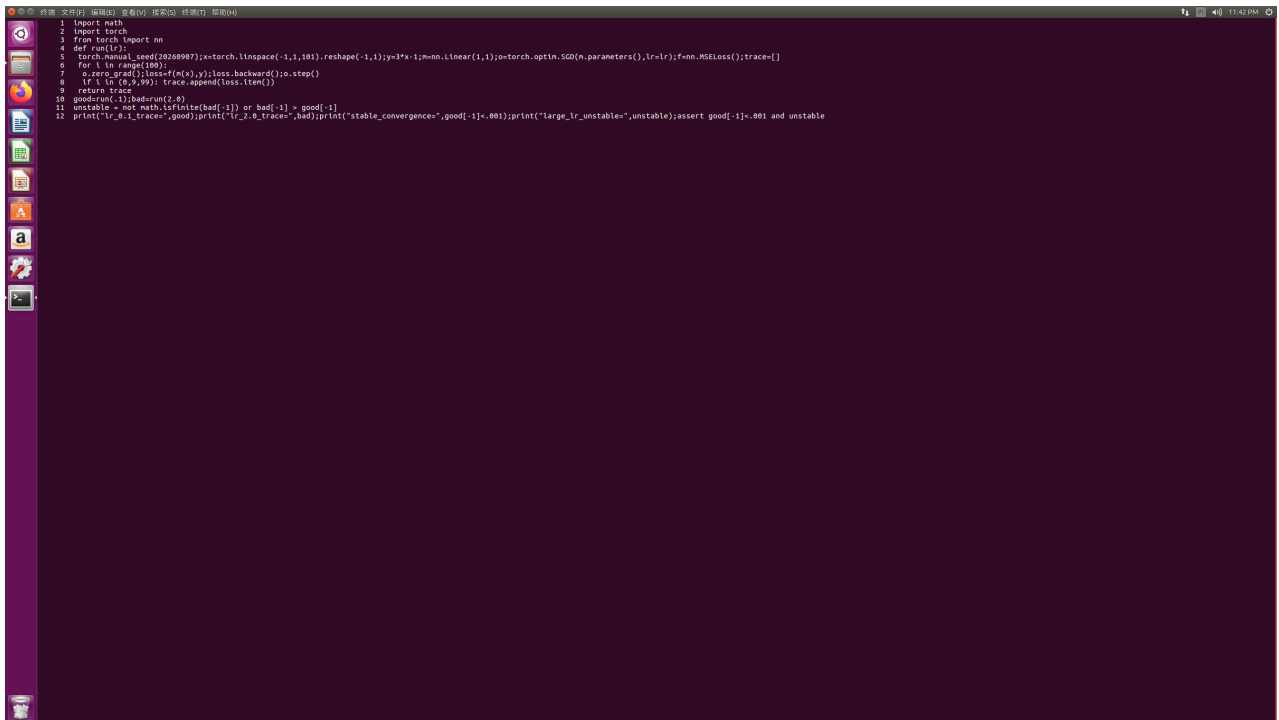


```
wheel_path=../q99/dist/q99-0.1.0-py3-none-any.whl
record_rows=7
verified_hashes=7
record_integrity=passed
```

图 24: 实例 9: 制品完整性通过

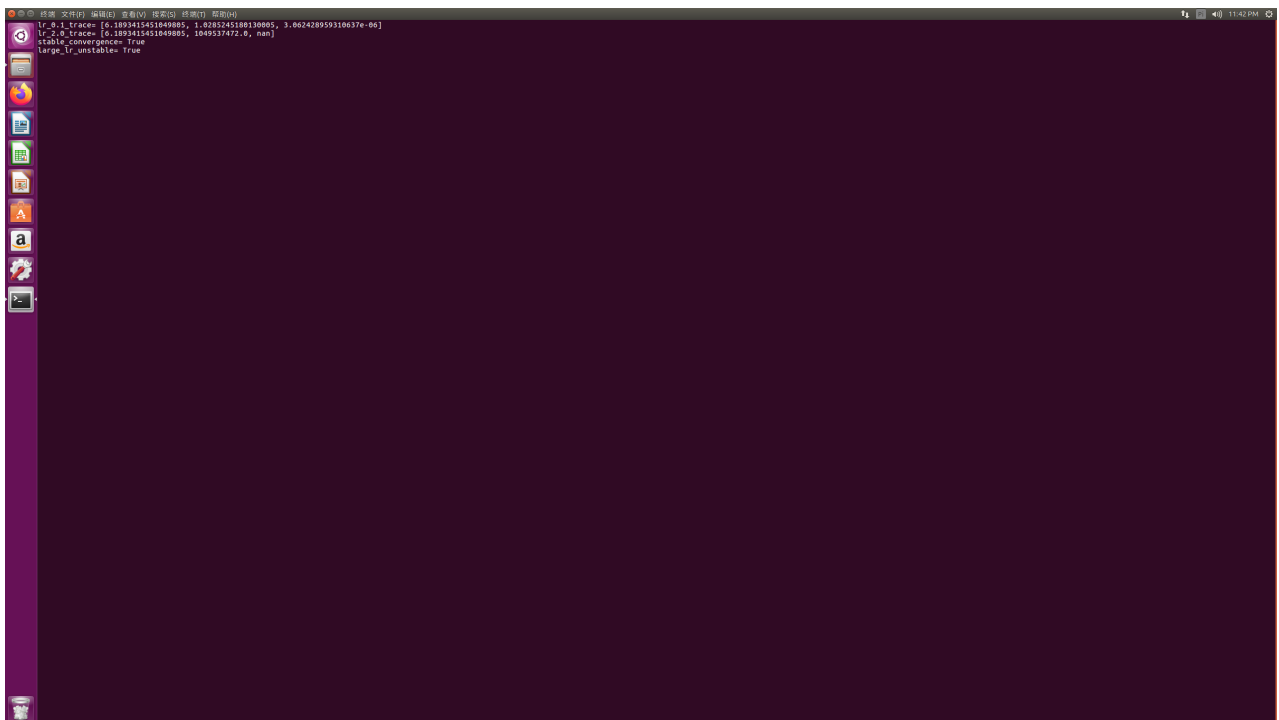
6.10 实例 10：学习率稳定性

学习率 0.1 在 100 轮收敛至 $3.06e-6$ ；2.0 迅速发散至 NaN。最初 50 轮未达到阈值，增加轮数；NaN 改用 `isfinite` 判断。



```
1 import math
2 import torch
3 from torch import nn
4 def run(lr):
5     torch.manual_seed(20268987);x=torch.linspace(-1,1,101).reshape(-1,1);y=3*x-1;model=nn.Linear(1,1);optimizer=torch.optim.SGD(model.parameters(),lr=lr);loss_fn=nn.MSELoss();trace=[]
6     for i in range(100):
7         optimizer.zero_grad();loss=f(model(x),y);loss.backward();optimizer.step()
8         if i in (0,9,99): trace.append(loss.item())
9     return trace
10 good=run(0.1);bad=run(2.0)
11 unstable = not math.isfinite(bad[-1]) or bad[-1]> good[-1]
12 print("lr_0_1_trace:",good);print("lr_2_0_trace:",bad);print("stable_convergence=",good[-1]<=0.001);print("large_lr_unstable=",unstable);assert good[-1]<=0.001 and not unstable
```

图 25: 实例 10：学习率比较



```
lr_0_1_trace: [6.1893415451849885, 1.0285245180130805, 3.062428959310637e-06]
lr_2_0_trace: [6.1893415451849885, 1049537472.0, nan]
stable_convergence: True
large_lr_unstable: True
```

图 26: 实例 10：收敛与发散

7 GitHub 分阶段提交记录

课堂 9–10 题、11–12 题分别提交；扩展实例每两个一次 commit，最后报告单独提交。主要历史为 6d38a83、aed47bc、00690e7、aa2863e、48ed87a、3f7bf7f 和 a18e186。

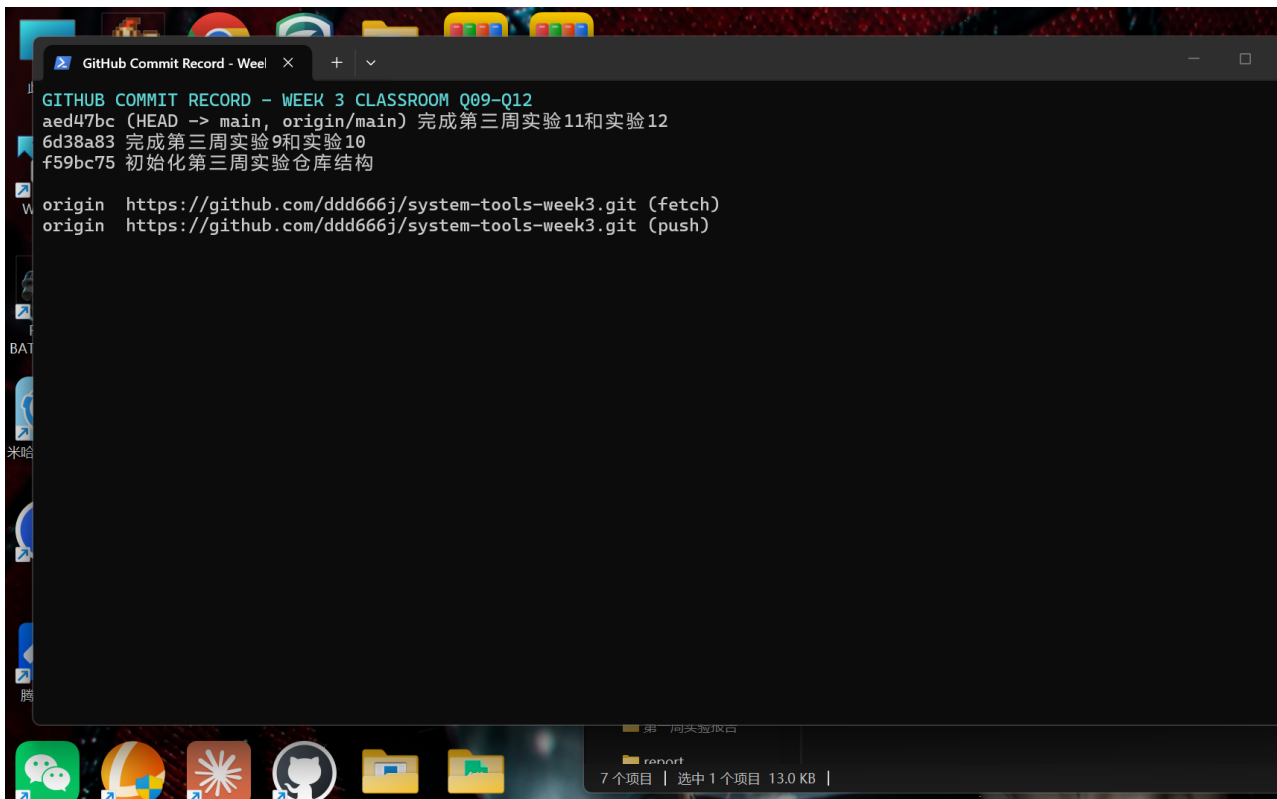


图 27: 课堂 9–12 题提交记录



图 28: 实例 1-2 提交记录

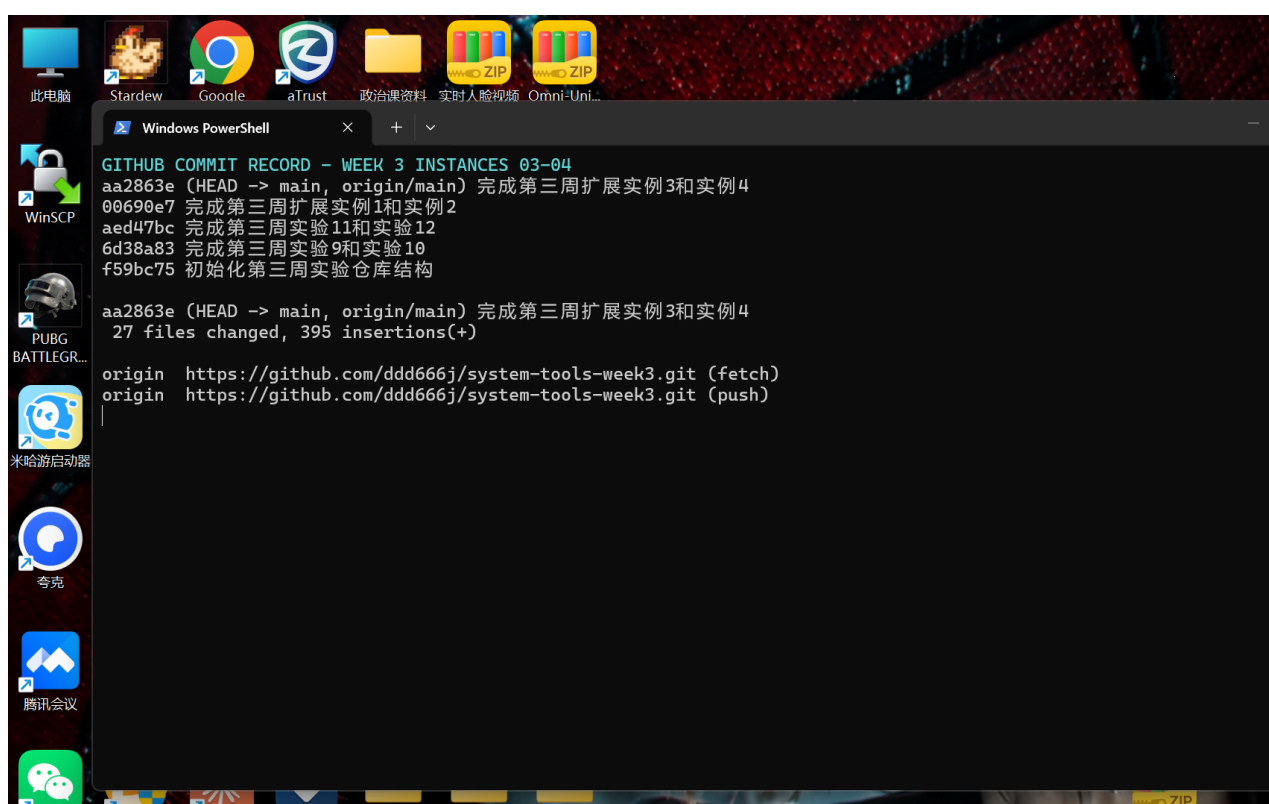
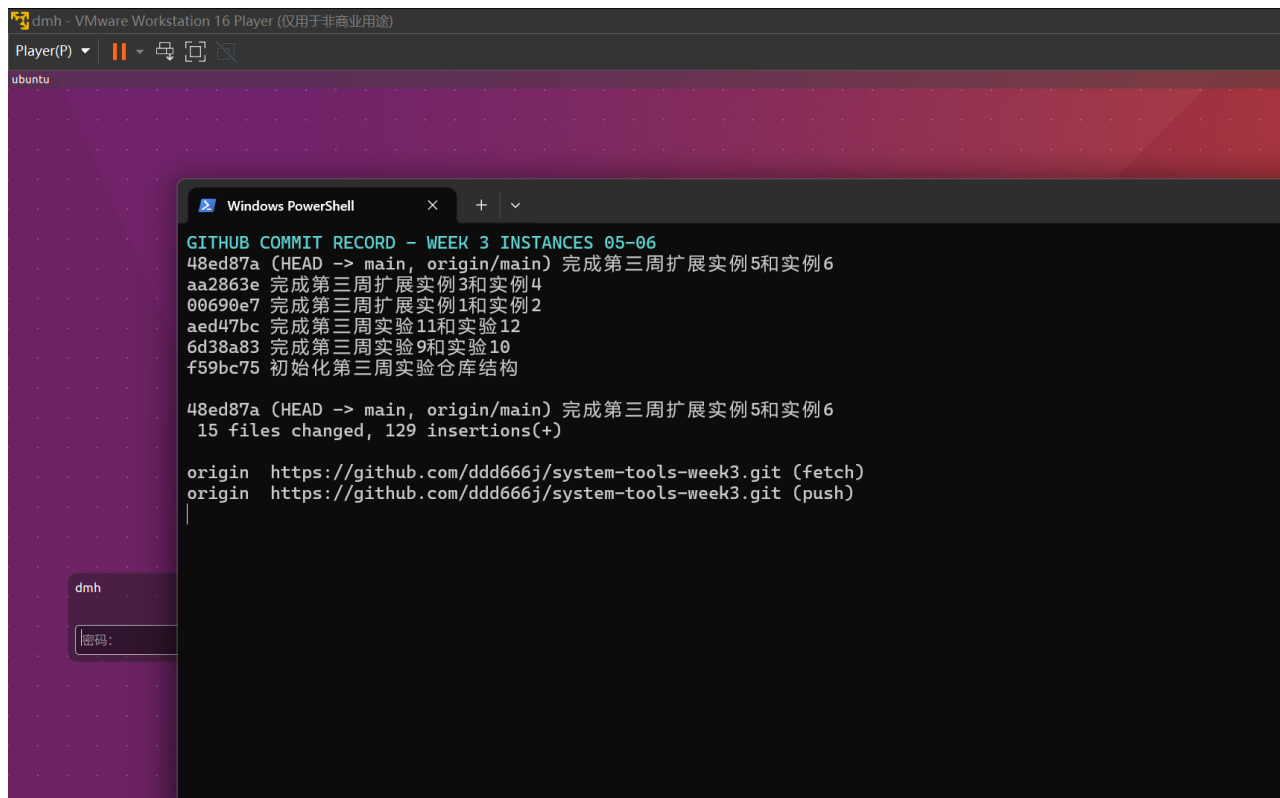


图 29: 实例 3-4 提交记录



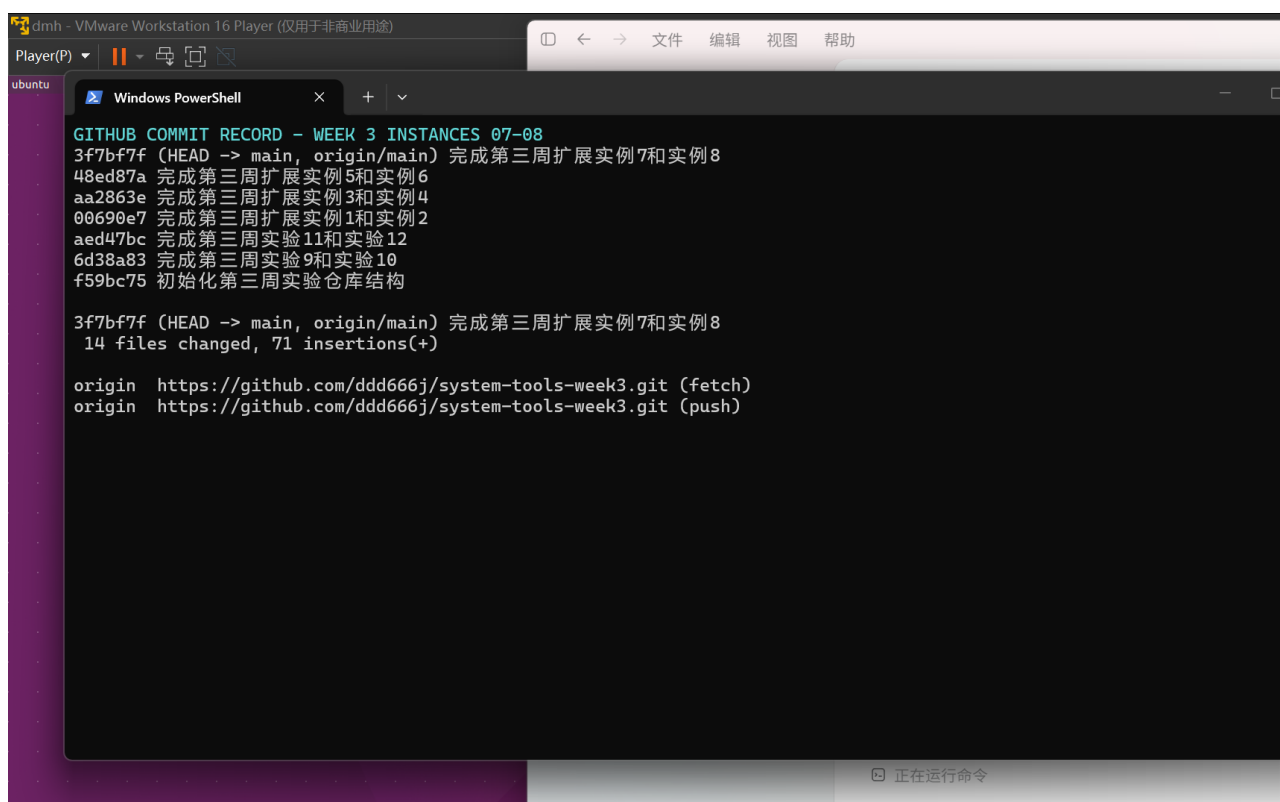
```
dmh - VMware Workstation 16 Player (仅用于非商业用途)
Player(P) | [Icons]
ubuntu

Windows PowerShell
GITHUB COMMIT RECORD - WEEK 3 INSTANCES 05-06
48ed87a (HEAD -> main, origin/main) 完成第三周扩展实例5和实例6
aa2863e 完成第三周扩展实例3和实例4
00690e7 完成第三周扩展实例1和实例2
aed47bc 完成第三周实验11和实验12
6d38a83 完成第三周实验9和实验10
f59bc75 初始化第三周实验仓库结构

48ed87a (HEAD -> main, origin/main) 完成第三周扩展实例5和实例6
15 files changed, 129 insertions(+)

origin https://github.com/ddd666j/system-tools-week3.git (fetch)
origin https://github.com/ddd666j/system-tools-week3.git (push)
```

图 30: 实例 5-6 提交记录



```
dmh - VMware Workstation 16 Player (仅用于非商业用途)
Player(P) | [Icons]
ubuntu

Windows PowerShell
GITHUB COMMIT RECORD - WEEK 3 INSTANCES 07-08
3f7bf7f (HEAD -> main, origin/main) 完成第三周扩展实例7和实例8
48ed87a 完成第三周扩展实例5和实例6
aa2863e 完成第三周扩展实例3和实例4
00690e7 完成第三周扩展实例1和实例2
aed47bc 完成第三周实验11和实验12
6d38a83 完成第三周实验9和实验10
f59bc75 初始化第三周实验仓库结构

3f7bf7f (HEAD -> main, origin/main) 完成第三周扩展实例7和实例8
14 files changed, 71 insertions(+)

origin https://github.com/ddd666j/system-tools-week3.git (fetch)
origin https://github.com/ddd666j/system-tools-week3.git (push)
```

图 31: 实例 7-8 提交记录

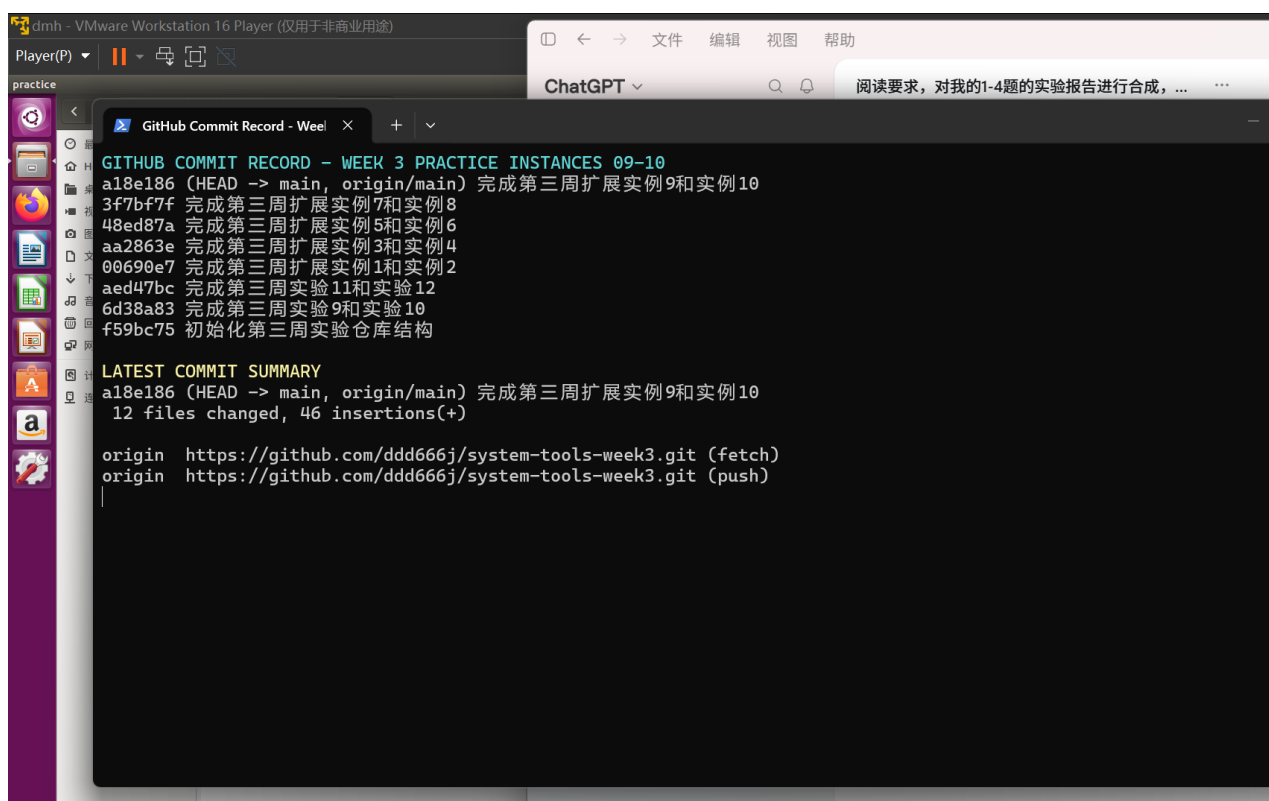


图 32: 实例 9-10 提交记录

8 综合结论与错误反思

本周形成“构建制品-干净安装-失败测试-最小修复-协作说明-CPU 训练-版本提交”的完整证据链。主要错误包括隔离构建网络失败、Shell 分组语法错误、训练轮数不足以及 NaN 不能普通比较。处理原则是保存失败现象、解释根因、实施最小修正并重新验收。实验说明软件交付不仅需要代码正确，还需要制品可验证、沟通可执行、训练可复现和历史可追踪。